

A 3.2.2 Entity relationship diagrams



A 3.2.2 Entity relationship diagrams

ERDs are crucial for creating well-structured databases designed for specific applications. They illustrate relationships between data entities and use cardinality/modality to define these relationships.

Introduction to Entity Relationship Diagrams (ERDs)

In the world of computer science, data is king. But raw data, like a disorganised pile of books, is useless. Databases provide the structure to organise and access data efficiently. Entity Relationship Diagrams (ERDs) are visual tools that help us design these databases effectively.

An **ERD** is a type of diagram that illustrates how "entities" (things or concepts about which we want to store information) relate to each other within a system. Think of it as a blueprint for your database.

The Significance of ERDs in Database Design

Entity Relationship Diagrams are crucial for several reasons:

Clarity and Communication: ERDs provide a clear, visual representation of the database structure. This makes it easier for developers, database administrators, and even non-technical stakeholders to understand the data requirements and how different parts of the system connect.

Organization and Efficiency: By mapping out entities and their relationships, ERDs help us design databases that are organized and efficient. This prevents data redundancy (storing the same information multiple times), ensures data integrity (accuracy and consistency), and makes data retrieval faster.

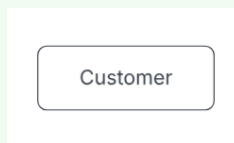
Tailoring to Specific Applications: Every application has unique data needs. ERDs allow us to design databases that are specifically tailored to the application's requirements. For example, a

database for an online store will have different entities and relationships than a database for a social media platform.

Foundation for Database Implementation: The ERD serves as a blueprint for the database's physical implementation in a Database Management System (DBMS) like MySQL, PostgreSQL, or Oracle.

Entities and Attributes

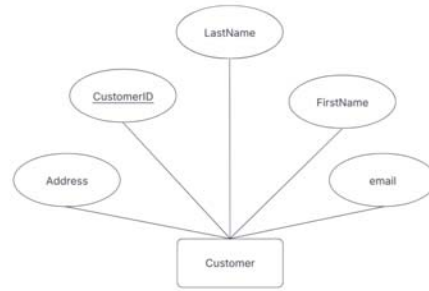
Entity: An entity is a "thing" or "object" in the real world that we want to store information about. Entities can be tangible (e.g., a customer, a product, an order) or intangible (e.g., a course, an event, a concept). In an ERD, entities are typically represented by rectangles.



Attribute: An attribute is a characteristic or property of an entity. It's a piece of information that describes the entity. For example, the entity "Customer" might have attributes like "CustomerID," "FirstName," "LastName," "Address," and "Email." Attributes are often represented by ovals.



Key Attribute: An attribute (or combination of attributes) that uniquely identifies each instance of an entity. This is called a primary key. For example, "CustomerID" is likely to be the primary key for the "Customer" entity. Primary keys are usually underlined in an ERD.



Composite Attribute: An attribute that can be further divided into smaller parts. For example, "Address" can be broken down into "Street," "City," "State," and "Zip Code."

Multi-valued Attribute: An attribute that can have multiple values for a single instance of an entity. For example, a person can have multiple "Phone Numbers."

Derived Attribute: An attribute that can be calculated or derived from other attributes. For example, "Age" can be derived from "Date of Birth."

Relationships Between Data Entities

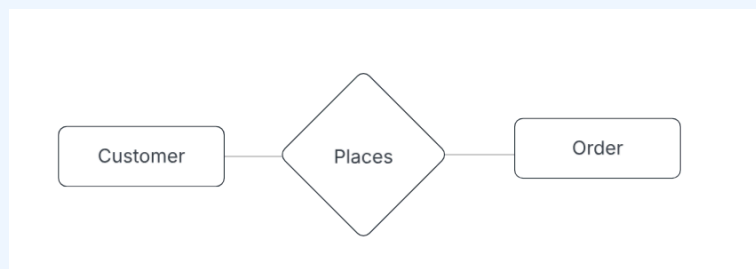
Relationships are the connections between entities. They describe how entities are associated with each other. In an ERD, relationships are represented by diamond shapes and are labeled with verbs that describe the connection.

Example: In a database for an online store, there might be a relationship between the "Customer" entity and the "Order" entity. This relationship could be named "Places" (a customer places an order).

Types of Relationships:

Unary Relationship (Recursive): A relationship between instances of the same entity type. For example, an "Employee" can be managed by another "Employee."

Binary Relationship: A relationship between instances of two different entity types. This is the most common type of relationship. For example, a "Customer" "Places" an "Order."



Ternary Relationship: A relationship between instances of three different entity types. For example, a relationship between "Doctor," "Patient," and "Appointment."

Cardinality and Modality in Defining Relationships

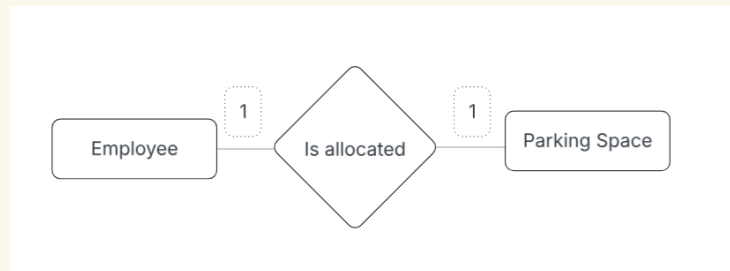
Cardinality and modality are crucial concepts for defining the constraints and rules of relationships in ERDs.

Cardinality

Cardinality refers to the number of instances of one entity that can (or must) be associated with another entity. It describes the "quantity" of the relationship.

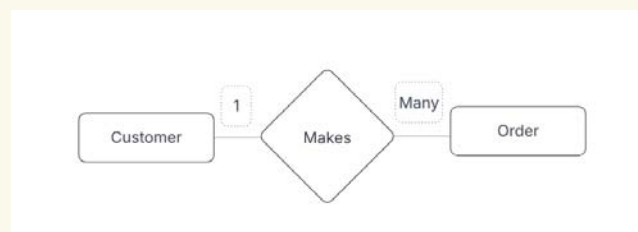
One-to-One (1:1): Each instance of entity A is associated with at most one instance of entity B, and each instance of entity B is associated with at most one instance of entity A.

For example, in some systems, each "Employee" is assigned one "Parking Space," and each "Parking Space" is assigned to one "Employee."



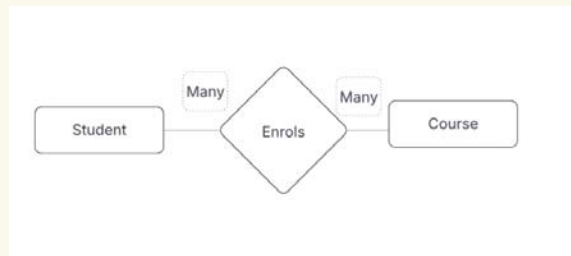
One-to-Many (1:M): Each instance of entity A can be associated with zero, one, or many instances of entity B, but each instance of entity B is associated with exactly one instance of entity A.

For example, a "Customer" can place many "Orders," but each "Order" is placed by only one "Customer."



Many-to-Many (M:N): Each instance of entity A can be associated with zero, one, or many instances of entity B, and each instance of entity B can be associated with zero, one, or many instances of entity A.

For example, a "Student" can enroll in many "Courses," and each "Course" can have many "Students." (Many-to-many relationships are often resolved by introducing a "junction" or "associative" entity).



Modality (Optionality)

Modality refers to whether the existence of an instance of one entity depends on the existence of an instance of another entity. It describes the "necessity" of the relationship.

Mandatory (Required): An instance of an entity must be related to an instance of the other entity. Represented by a solid line or a symbol indicating "one or more."

Optional: An instance of an entity may or may not be related to an instance of the other entity. Represented by a dashed line or a symbol indicating "zero or one."

ERD Notation

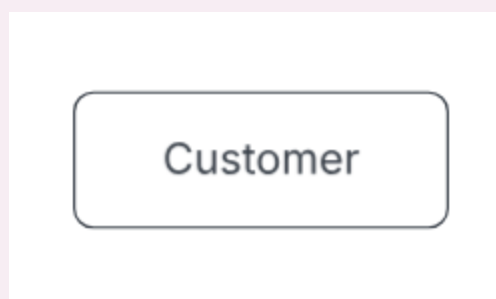
There are two different notations used for constructing ERDs, which are:

Chen Notation

One of the earliest notations, it uses rectangles for entities, ovals for attributes, diamonds for relationships, and lines to connect them.

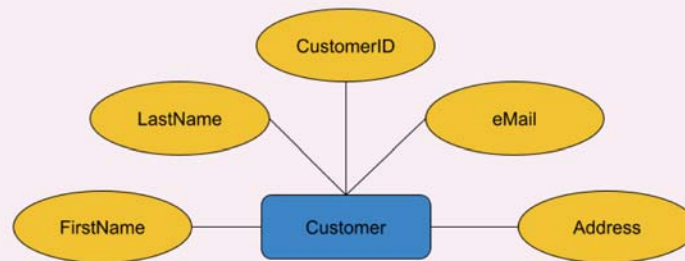
Entities

An entity is represented by a rectangle which contains the entity's name



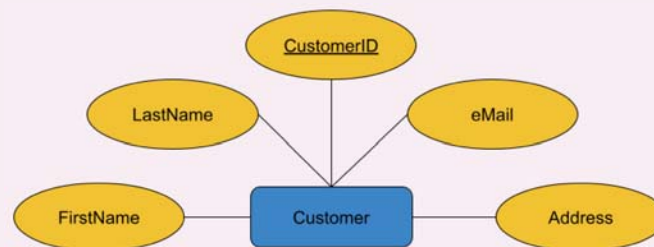
Attributes

In the Chen notation, each attribute is represented by an oval containing attribute's name:

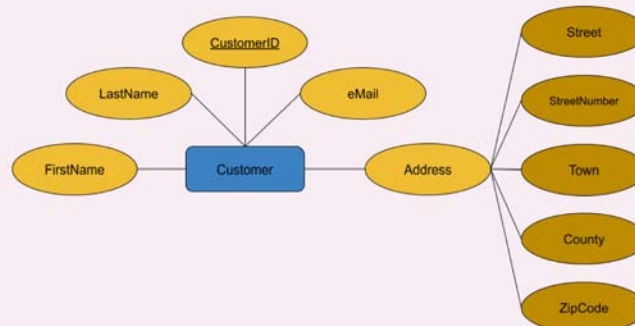


There are specific types of attributes:

1. **Key Attribute:** This is an attribute that us the primary key for the record. A primary key is a field that can uniquely identify a record. For example the CustomerID is a unique code used to identify individual customers in the database and no two customers can have the same CustomerID. A key attribute is underscored.

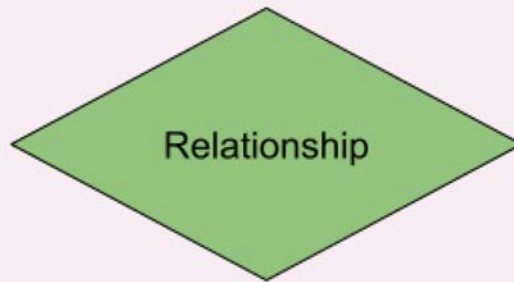


2. **Composite attributes:** Some attributes can be further subdivided into smaller parts. For example the "address" attribute can be. subdivided into street name street number town county country zip code etc.



Relationships

In Chen notation a relationship is represented by a diamond which contains the relationship's name.

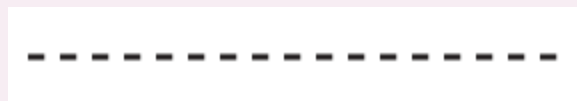


Optionality of a relationship

A mandatory relationship is represented by a solid line



An optional relationship is represented by a dashed line

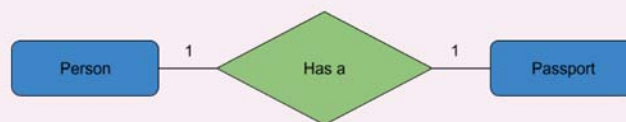


Cardinality

The degree of relationship is represented by any of the following:

one-to-one

A Person can have at most one Passport, and a Passport belongs to at most one Person.



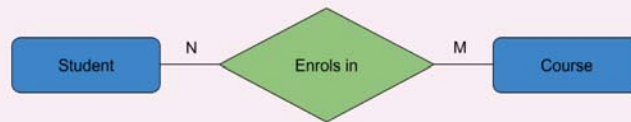
one-to-many

An Author can write many Books, but each Book is written by at most one Author.



many-to-many

A Student can enroll in many Courses, and a Course can have many Students



Crow's Foot Notation

A more common notation, especially for relational databases. It uses rectangles for entities, attributes listed inside, and special symbols at the ends of the lines representing relationships to show cardinality and modality (the "crow's feet").

Entities

Table Name		
Key	Field Name	DataType

Relationships

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many

Example: Online shop

Entities:

Customer
Order

Product

Attributes:

Customer: CustomerID (PK), FirstName, LastName, Email, Phone, Address

Order: OrderID (PK), OrderDate, Quantity, TotalAmount, CustomerID (FK), ProductID (FK)

Product: ProductID (PK), Name, Description, Price, Stock Amount

Relationships:

Customer places Order (One-to-Many, Order is mandatory for Customer)

Order contains Product (One-to-Many, Order contains one or more products)

Crows Foot notation

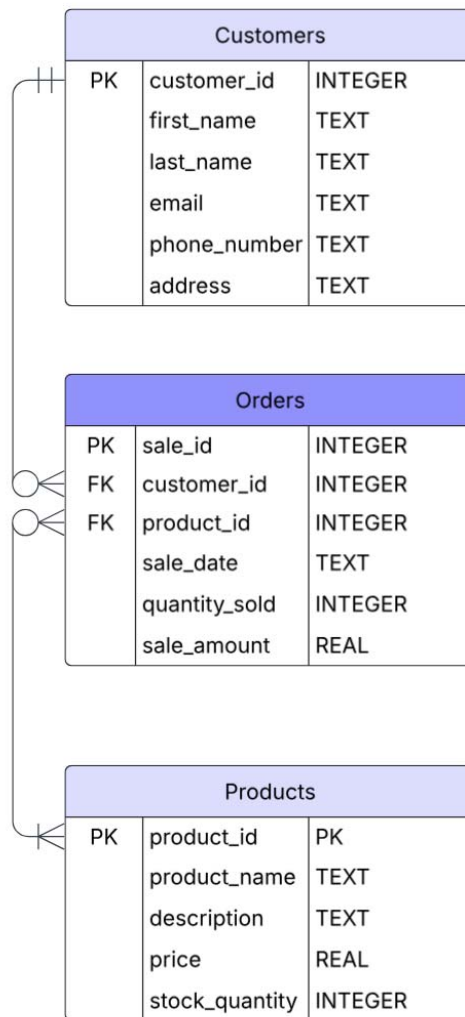
Notice the primary key and foreign key allocation in this diagram. We will look at these later in A 3.2.4 but for now you should understand that these signify fields within an entity that are unique (not duplicated) and can be used to identify each record in a database.

When deciding on the relationships you need to think in terms of the relationship of each entity to the other entity in both directions.

For example in the relationship between the customer and the order.

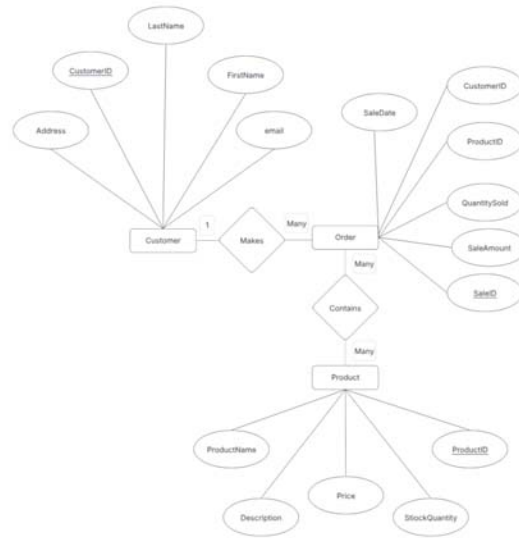
Customer to Order: There must be at least one customer and there can only be one customer related to an order.

Order to Customer: A customer may have many order but there may also be a customer with zero orders.



Chen notation

Notice that the Chen notation is simpler in some ways than the Crow's foot diagram but this type of diagram can be very useful when first thinking about the structure and relationships within a database.



Key Considerations When Constructing ERDs

Understand the Requirements: The most crucial step is to thoroughly understand the data requirements of the system you are designing. Talk to stakeholders, analyze documents, and clarify any ambiguities.

Identify Entities and Attributes: Carefully identify the entities that need to be represented in the database and the attributes that describe them.

Define Relationships: Determine how the entities are related to each other. Choose appropriate relationship types, cardinality, and modality to accurately reflect the business rules.

Choose a Notation: Select a consistent ERD notation (Chen, Crow's Foot, etc.) and stick with it throughout the design process.

Refine and Iterate: ERD design is often an iterative process. You may need to refine your diagram as you gain a better understanding of the requirements or as the system evolves.

Normalization: Consider database normalization principles to reduce redundancy and improve data integrity.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.